UNIVERSITY OF MORATUWA

Department of Computer Science and Engineering

Feasibility Study

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing

Project ID(PID)	11
Group Number	25
Mentor	Mr. Bhanuka Siriwardhana

Group Members

Registration No.	**Name**
230667F	Virusan T.
230670H	Vishula J.
230674A	R.P.M.Vithanage

Table of Contents

Individual Contributions **3**
1. Introduction **4**
1.1 Overview of the Project 4
1.2 Objectives of the Project 5
1.3 Need of the project 6
1.4 Overview of Existing Systems and Technology 6
1.5 Scope of the Project 6
1.6 Deliverables 7
2. Feasibility Study **8**
2.1 Financial Feasibility 8
2.1.1 Development Cost Estimate 8
2.1.2 Deployment Cost Estimate 8
2.1.3 Financial Benefits (Return on Investment) 9
2.2 Technical Feasibility 9
2.2.1 Application Architecture 9
2.2.2 Technical Justification for selected technologies 10
2.3 Resource and Time Feasibility 13
2.3.1 Hardware requirements 13
2.3.2 Software requirements 13
2.3.3 Human Resource Feasibility 14
2.3.4 Time Feasibility 14
2.4 Risk Feasibility 15
Risk Assessment Summary 16
2.5 Social / Legal Feasibility 16
3.Considerations **17**
3.1 Performance 17
3.2 Scalability 17
3.3 Security 17
3.4 Reliability 17
3.5 Usability 17
3.6 Maintainability 17
3.7 Availability 18
4.References **18**

Individual Contributions

Index No.	Name	Contribution
230667F	Virusan T.	• Introduction • 2.1 Financial Feasibility • 2.3 Resource and Time Feasibility • 2.5 Social / Legal Feasibility • 3 Considerations
230670H	Vishula J.	• Introduction • 2.3 Resource and Time Feasibility • 2.2.1 Application Architecture • 2.2.2 Technical Justification • 2.3 Resource and Time Feasibility
230674A	R.P.M.Vithanage	• Introduction • 2.1 Financial Feasibility • 2.2.1 Application Architecture • 2.2.2 Technical Justification • 2.4 Risk Feasibility

1. Introduction

1.1 Overview of the Project

This project is about a web based B2B marketplace for second hand vehicles. The system allows vehicle dealers to upload inventories in bulk using CSV or JSON files along with ZIP archives of vehicle images. Each vehicle record includes details like make, model, price, mileage, fuel type , associated images etc.

Uploaded inventories go through an automated Extract, Transform, Load (ETL) pipeline. In the Extract stage, vehicle data and images are uploaded and stored in Amazon S3 [8]. In the Transform stage, serverless backend services validate records, standardize vehicle attributes, map images with listings and create proper metadata. In the Load stage, validated records are stored in PostgreSQL [6], while searchable metadata and semantic embeddings are indexed using PostgreSQL extensions such as pgvector and pg_trgm [7].

The backend will be implemented as microservices using AWS Lambda, API Gateway, SQS and Docker based Lambda container images. These images will be stored in Amazon ECR and executed by AWS Lambda when triggered by API requests or asynchronous events.

Buyers and guests can browse vehicles with standard filters such as make, model, price, mileage, fuel type and transmission type. Registered buyers can also save favourite vehicles and search using natural language queries such as, “I need an automatic SUV under 20 million LKR.” The search module interprets the query, extracts important data and returns relevant vehicles through filtering and semantic search.

Inputs	Outputs
Dealer, buyer and admin registration details	Validated and processed vehicle listings
CSV or JSON vehicle inventory files	Optimized vehicle images
ZIP files containing vehicle images	Traditional and intelligent search results
Vehicle search requests	Dealer inventory dashboard
Dealer inventory updates	Upload status reports and error reports
	Email notifications
	Synthetic datasets for testing and development

1.2 Objectives of the Project

The objectives of this project are to:

- To design a scalable, B2B architecture for bulk vehicle data ingestion and processing.
- To implement an asynchronous Automated ETL (Extract-Transform-Load) Pipeline that validates, transforms, standardizes and enriches vehicle listings with consistent metadata.
- To provide traditional filter based and intelligent natural language vehicle search.
- To develop a dealer dashboard for managing their inventories.
- To deploy the system in local Docker environments and AWS cloud infrastructure.
- To establish a CI/CD process pipeline with automated testing and deployment.

1.3 Need of the project

Many car dealers still upload vehicle details manually, which takes a lot of time and can cause errors. Vehicle information may be incomplete or inconsistent. Also most locally existing websites only allow buyers to search vehicles using filters. They cannot easily search using normal English sentences.This project targets to solve these problems by providing bulk uploads, automatic data processing, intelligent search and cloud based deployment. It helps dealers manage their inventory more easily and helps buyers find suitable vehicles efficiently. Although existing international products solve some of these issues they are expensive for small scale car dealers.

1.4 Overview of Existing Systems and Technology

There are several online vehicle marketplaces such as AutoTrader [1], Cars.com [2], ikman.lk [3], and Riyasewana [4] that allow users to search and view vehicle listings .However, the international platforms among these are designed for large businesses and may not provide affordable solutions for smaller dealerships. Out of these, the local platforms may focus mainly on traditional search methods and have the manual entry method.Our project will use modern cloud technologies and automation to improve inventory management and provide a better search experience.

1.5 Scope of the Project

The proposed system will include four main user roles.

Dealer - Register/login, Upload vehicle inventories in bulk (CSV/JSON) along with image assets, Track processing pipeline status, Manage active listings via an inventory dashboard, Receive automated email summaries upon upload completion.

Buyer - Register/log in, Browse the vehicle catalogue, Search using Natural Language or filters, View details of vehicles, Save favourite information

Guest - Browse available vehicles, Search vehicles using filters or natural language queries, View details of vehicles, Register or log in to access additional features

Administrator - Manage users (dealers and buyers), Monitor upload jobs and processing pipelines, View system logs and reports, Manage the platform

The Automated ETL Pipeline - Data validation, Data normalization, Image to vehicle mapping, Image resizing, compression and format conversion (if needed), Metadata generation which are used for search, Storage of validated listings, Reporting invalid records, Asynchronous job processing and status tracking

1.6 Deliverables

- A web based second hand vehicle marketplace
- A Dealer dashboard
- An Admin dashboard
- A buyer and guest vehicle search interface
- Bulk CSV/JSON inventory uploading functionality
- An Automated ETL Pipeline
- Vehicle image upload and optimization module
- Filter based and natural language based search
- Upload status reports and email notifications
- Docker based local development environment
- AWS cloud deployment
- CI/CD pipeline configuration
- Synthetic vehicle data generation tool for testing
- Technical documentation and user documentation

2. Feasibility Study

2.1 Financial Feasibility

2.1.1 Development Cost Estimate

The project is based on open source tools and local development environments to keep initial costs at zero.

Cost Area	Description	Estimated Cost
Personnel cost	No direct developer payment is required since this is an academic project. In a commercial implementation, personnel costs would include for frontend and backend development , UI/UX design, testing, DevOps and project management	LKR 0
Software licences	React, NestJS, PostgreSQL, Docker, Terraform, Sharp, pgvector, pg_trgm and Hugging Face embedding models are open source or freely available.	LKR 0
Version control and CI/CD	GitHub will be used for version control and GitHub Actions will be used for automated testing and deployment.	LKR 0
Local development environment	The frontend, backend, database and ETL worker can be run locally using Docker. This reduces the need for continuous cloud usage during development.	LKR 0
Testing data	A synthetic vehicle data generation tool will be used to create test CSV/JSON files. This avoids the need to purchase external datasets.	LKR 0

2.1.2 Deployment Cost Estimate

The production system will run on Amazon Web Services (AWS) using a pay-as-you-go model. This allows the platform to start small and scale as usage increases. All these services will fall under the AWS Free Tier. During development, expected usage remains within or close to the AWS Free Tier limits.

2.1.3 Financial Benefits (Return on Investment)

The system's financial value could be understood by comparing current manual data entry costs with the expected automated system.

The main benefits include:

1. **Labour Cost Savings:** Manual data entry is slow. Bulk uploading shifts the workload to background processes.

$$Monthly\ labour\ saving\ =\ Number\ of\ listings\ per\ month\ x\ Time\ saved\ per\ listing\ x\ Staff\ hourly\ cost$$

2. **Error Reduction Savings:** Manual entry causes mistakes (wrong prices, missing details) that require paid staff time to fix. The automated pipeline validates and corrects data instantly.

$$Monthly\ Rework\ Savings\ =\ L \times E \times T \times C$$

$L\ =\ Number\ of\ listings$
$E\ =\ Expected\ error\ rate$
$T\ =\ Rework\ time\ per\ error\ (hours)$
$C\ =\ Staff\ hourly\ cost$

3. **Total Operational Savings:**

$$Total\ monthly\ saving\ =\ Labor\ saving\ +\ Rework\ saving$$

Even a single dealership will see meaningful annual savings by reducing manual work and increasing inventory publishing speed. Therefore, the project is financially feasible and beneficial for small and medium scale dealers.

2.2 Technical Feasibility

2.2.1 Application Architecture

The system will use a serverless microservices architecture using AWS Lambda. Instead of creating too many small services, the system will be divided into four main Lambda based microservices.

The planned microservices are:

Microservice	Main Responsibility
Auth & User Service	Registration , Login , JWT Authentication , User Roles
Marketplace Service	Dealer Profiles , Vehicle Listings , Vehicle

	Images , Buyer favourites , filter search and natural language search
Ingestion/Upload & ETL service	Bulk CSV / JSON and ZIP uploads , upload jobs , validation , normalization , image processing and rejected records
Admin & Notification Service	Admin monitoring , reports , audit logs , upload status , SES Email notifications

Auth & User Service is kept separate because it handles sensitive security related data such as passwords. It should not be mixed with marketplace logic.

Marketplace Service owns the main vehicle data. Search is included inside Marketplace Service because listings and search use the same vehicle tables and queries. Separating search would require another service to either access Marketplace data directly or maintain a copy, which would increase complexity.

Ingestion / Upload & ETL Service is separate because bulk upload processing is asynchronous and a heavy task. It may handle large CSV/JSON files, ZIP files, image processing and embedding(vector) generation. Keeping it separate prevents heavy upload jobs from slowing down general interactions.

Admin & Notification Service is separate because it is used by administrators and system events. Admin reports, audit logs and email notifications have different access levels and runtime behaviour compared to buyer facing vehicle browsing.

2.2.2 Technical Justification for selected technologies

React - Frontend

React is selected because the project needs reusable UI components such as dashboards, search bars,upload forms and vehicle detail pages. When compared to plain HTML/CSS/JavaScript, React makes the UI easier to organize and change. It is also simpler and less heavy than frameworks like Angular. Also React has a huge community support.

NestJS with TypeScript - Backend

According to references Nodejs has been tested to handle many concurrent IO requests. This avoids thread blocking issues with Python based frameworks like FastAPI which when combined asynchronous and synchronous libraries.

Out of the common frameworks of Nodejs , NestJS is selected because it provides a clean structure for backend code using modules, controllers, services and guards. This helps organize backend logic clearly. NestJs is much recommended for enterprise level projects due to this architecture rather than Express framework which does not have this method and was built on javascript not typescript.

Eventhough Spring boot has same kind of architecture due to less complexity and easier configuration , NestJs is chosen

TypeScript helps to have a safer codebase as it catches errors related to data shape during compile time not in run time like javascript , therefore it will be more suitable for the project than javascript.

AWS Lambda - Runs Backend services

AWS Lambda is selected because the backend functions are event-driven. For example, API requests, upload events, ETL activities triggered by SQS can be handled by Lambda.Therefore no need to manage EC2 servers or ECS in AWS.

Amazon API Gateway

AWS API Gateway is selected because it acts as the entry point between the React frontend and Lambda services. It routes requests to the correct Lambda function.

Docker

Docker will be used as the platform to let the team work in a consistent local environment. It is also useful for packaging Lambda functions as container images.

Amazon ECR - Stores backend Docker images

Amazon ECR is selected because Lambda container images need to be stored in a container registry like docker hub. The team can build Docker images locally and push them to ECR and then deploy those images to AWS Lambda.

Amazon RDS PostgreSQL - Main database

In development, PostgreSQL will run locally using a Docker image or using native software, while deployment will use one Amazon RDS PostgreSQL instance. That database is expected to have one main database with separate service based schemas or tables for each microservice.

PostgreSQL is selected because the system has related data such as users, dealers, vehicles, favourites and upload jobs. It also supports pgvector like extensions allowing semantic search therefore no need for a separate search database.

Compared with a non relational database, PostgreSQL reduces data duplication and the amount of backend code needed to manage relationships and consistency of data.

Pgvector

pgvector is used because it allows vehicle listing vectors and buyer query vectors to be stored and compared inside PostgreSQL. This supports natural language search without needing a separate vector database.

Rejected the idea of using OpenSearch vector database/search engine as it needs a separate RDS instance which will increase the cost and complexity.

Pg_trgm

This is useful when users make spelling mistakes. It helps for similarity based text matching.

Hugging Face all-MiniLM-L6-v2 model

This already trained model is selected because it can convert vehicle descriptions and buyer search queries into vectors for semantic search. The same model will be bundled inside marketplace service and ETL service. The team does not need to train a custom ML model, which saves time and keeps the project feasible. Using an LLM would increase cost, latency and dependency on an external API, which is not ideal for the expected project scale.

S3

S3 is chosen because uploaded CSV/JSON files, processed vehicle images etc need to be stored efficiently. Large files should not be stored directly in PostgreSQL, instead, metadata and S3 file paths will be saved to PostgreSQL.

SQS

The system will not process large uploads directly inside the initial API request because it could make the user wait for a long time. Large files, ZIP extraction, image processing, validation and vector generation can take time. If everything is done in one request, the API may become slow. That is the reason to use a queue for this kind of application.

Using SQS allows the upload request to finish quickly. The dealer receives an upload job status and the ETL process continues in the background. This makes the system more reliable and scalable. SQS is preferred over other queueing options available because it is fully managed, serverless, low cost and directly works along with AWS Lambda. Other options such as RabbitMQ or Kafka would require more setup and maintenance.

SES

Since the system needs to send upload success or failure emails to dealers an email service is needed. AWS based SES is suitable because email notifications can be handled automatically and directly work with the rest of the technologies the project expects to work with.

CloudWatch

CloudWatch is selected because Lambda functions, API Gateway and ETL processing need logs for debugging and monitoring.

Terraform

It allows AWS infrastructure to be written as code. This makes the cloud setup repeatable and easier to manage. It also reduces errors that come along with manual configuration.

IAM

IAM is used as each AWS service needs controlled permissions. For instance, the Ingestion service needs access to S3 bucket and SQS, while the Admin & Notification service needs access to SES. IAM improves security by giving each service only the permissions it needs.

GitHub Actions

Automate testing, Docker image building, pushing images to ECR and deployment will be done by Github actions. This reduces manual deployment work and supports continuous integration.

Postman

Postman will be used to test API Gateway and Lambda endpoints before connecting them to the frontend.

2.2.3. Deployment Feasibility

During development, Docker Compose will be used to run the local environment consistently.

During deployment, the React frontend will be built as static files and hosted in Amazon S3 with CloudFront if needed. The backend microservices will be stored in AWS ECR. AWS Lambda will run these container images when triggered by API Gateway requests or SQS messages.

Frontend requests will first go through AWS API Gateway. API Gateway will route requests to the correct Lambda service, such as Auth, Marketplace, Ingestion / ETL or Admin & Notification. Uploaded CSV/JSON files and vehicle images will be stored in Amazon S3. Upload jobs will be placed in Amazon SQS and the Ingestion / ETL Lambda will process them asynchronously in the background.

Amazon RDS PostgreSQL will store application data using one database with separate service owned schemas. Amazon SES will send upload success or failure emails to dealers. Amazon CloudWatch will store logs and help monitor Lambda execution, API errors and ETL failures. IAM roles will be used to give each AWS service only the permissions it needs.

Terraform will be used to define and create the required AWS infrastructure, while GitHub Actions will automate testing, Docker image building, pushing images to ECR and deployment. This makes the deployment process repeatable and reduces manual configuration errors.

Therefore, this deployment approach is feasible because it uses a clear serverless deployment flow, supports local testing through Docker and avoids unnecessary server management during the prototype stage.

2.3 Resource and Time Feasibility

The project will be completed by a team of 3 undergraduate students within the allocated time.

2.3.1 Hardware requirements

- Personal computer with at least 8 GB RAM.
- At least 20 GB of free disk space for Docker and databases.
- Stable internet connection.

2.3.2 Software requirements

- Visual Studio Code
- Docker Desktop
- Node.js
- PostgreSQL
- Git
- Terraform
- Postman
- Modern web browser

2.3.3 Human Resource Feasibility

The project will be completed by a team of three CSE undergraduates. The work is divided into three manageable tracks:

- **Frontend:** User interfaces, dashboards and search inputs.
- **Backend:** Database design, security, search logic and API routes.
- **Data & DevOps:** ETL worker processing, image processing and cloud deployment.

2.3.4 Time Feasibility

The project timeline is feasible because the work is divided into clear stages: documentation, backend development, frontend development, cloud deployment, testing and final submission. The documentation phase starts with the project proposal, feasibility document, project schedule and then requirement gathering, SRS and design document. This ensures that the main requirements, architecture, database structure, and API design are planned before the full system is completed.

But due to time constraints the team expects to start development during the design phase after the initial project setup. GitHub and database setup are completed first, followed by the

Auth & User Service, Marketplace Service, local API gateway, Ingestion / ETL Service and Admin & Notification Service. Unit testing starts together with backend development, so each module can be tested while it is being implemented rather than waiting until the end of the project.

Frontend development is planned after the main backend services have started. The dealer, buyer and admin interfaces are developed first and then the frontend is connected with the API Gateway endpoints. This order is suitable because the frontend integration depends on the backend APIs being available.

Cloud deployment is scheduled after the major backend and frontend development work. During this stage, Terraform scripts, AWS API Gateway configuration and the CI/CD pipeline are completed. This allows the team to deploy the Lambda based backend services and connect them with AWS services such as RDS, S3, SQS, ECR, SES and CloudWatch.

Testing is not kept only for the final stage. Unit testing begins during development, component testing is carried out after major components are ready .Integration testing is done after deployment. UAT is planned near the end to verify that the system works from the user's point of view.

Therefore, the schedule is realistic because features are planned in a logical order. Advanced features such as payments, live bidding, and advanced recommendation systems are excluded from the first version, which helps keep the project achievable within the given time period.

2.4 Risk Feasibility

The project contains several risks related to data quality, processing reliability, search accuracy, cloud cost, security and project scope. However, these risks can be managed through proper design and mitigation strategies.

Risk	Description	Mitigation Strategy
Poor quality uploaded data	Dealers may upload files with empty fields, wrong formats, invalid prices or unmatched images	Before saving it, the ETL pipeline validates all fields. Broken rows are rejected and displayed in the dealer dashboard as a clear error report.
ETL failure	Large uploads may not complete.	AWS SQS will handle background jobs with retry support. Failed jobs will be logged and job status will be updated correctly.
Search inaccuracy	Buyers may receive irrelevant search results.	The system will combine exact filters with semantic search. Standard filters remain available as a backup option.

Cloud cost Over run	Not able to complete the project with the budget given	Docker will be used for local testing to reduce cloud usage. AWS billing alerts and Terraform resource limits will be used.
Data security issues	Sensitive data can be accessed by unauthorised users.	JWT authentication, role based access control, secure password hashing, HTTPS communication will be implemented.
Image processing issues	Some images may be corrupted, too large or not matched properly to vehicle records	Image validation and processing rules will be added. Unsupported or corrupted images will be rejected.
Network Failure	Temporary internet or AWS connectivity issues may affect access to cloud services.	Failed uploads and ETL jobs will be retried where possible and AWS service logs will be monitored via CloudWatch.
Team member unavailability	Unexpected absence delays specific development tasks.	Common and shared technical documentation will be used. Git version control will ensure any member can pick up pending tasks.
Deployment or integration errors	Problems may occur when connecting frontend, backend, database, API Gateway, Lambda, S3, SQS and RDS.	Testing locally using docker. Use Terraform, GitHub Actions, Postman, and CloudWatch logs to identify and fix issues. Also seek help from mentor

Risk Assessment Summary

Risk	Likelihood	Impact Severity	Overall Risk Severity
Poor quality uploaded data	High	Medium	High
ETL processing failure	Medium	High	High
Search inaccuracy	Medium	Medium	Medium
Cloud cost overrun	Medium	High	High
Data security threats	Medium	High	High
Team member unavailability	Medium	Medium	Medium
Network failure	Medium	Medium	Medium

Image processing issues	Medium	Medium	Medium
Deployment or integration errors	Medium	High	High

2.5 Social / Legal Feasibility

The proposed system provides an efficient digital platform for vehicle dealers and buyers, reducing manual effort in inventory management.

User information will be securely stored using hashed passwords and secure authentication mechanisms. The system will only process vehicle information uploaded by authorized dealers. No copyrighted vehicle images or personal information from third parties will be collected without permission.

Dealers will be responsible for ensuring that the vehicle details and images they upload are accurate and legally allowed to be published. The initial version of the system will not include payments, ownership transfers, vehicle financing, insurance processing etc.This reduces legal and regulatory complexity.

The project will be built according to standard software engineering practices regarding data privacy, security and ethical use of user information.

3.Considerations

In the development of the proposed system, several important software quality attributes will be considered.

3.1 Performance

The system should be able to efficiently handle large inventory uploads through asynchronous SQS based ETL processing and provide fast search responses through proper indexing

3.2 Scalability

The application should support increasing numbers of dealers, vehicle listings and concurrent users making use of cloud native architecture.

3.3 Security

User authentication will be built using JWT based authentication with role based authorization. Passwords will be securely hashed before being stored.Service owned schemas will also help maintain logical data boundaries.

3.4 Reliability

The ETL pipeline should ensure that invalid records do not affect valid uploads. Failed jobs should be logged and recoverable through retry methods.

3.5 Usability

The platform should provide a user friendly interface for dealers to manage inventories and for buyers to search vehicles using traditional filters as well as natural language queries.

3.6 Maintainability

The system will be a serverless microservices structure with four main services. This keeps the project manageable, but still separates responsibilities by security, data ownership and user audience. Infrastructure will be managed using Terraform and deployment will be automated using GitHub Actions.

3.7 Availability

Cloud deployment on AWS will provide the application with availability and can recover from service failures with minimal downtime.

4.References

[1] AutoTrader, “AutoTrader – Buy & Sell Cars.” [Online]. Available: https://www.autotrader.com/.

[2] Cars.com, “Cars for Sale – New and Used Cars.” [Online]. Available: https://www.cars.com/.

[3] ikman.lk, “Vehicles for Sale in Sri Lanka.” [Online]. Available: https://ikman.lk/.

[4] Riyasewana, “Vehicles for Sale in Sri Lanka.” [Online]. Available: https://riyasewana.com/.

[5] NestJS, “NestJS Documentation.” [Online]. Available: https://docs.nestjs.com/.

[6] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/.

[7] pgvector, “Vector Similarity Search for PostgreSQL.” [Online]. Available: https://github.com/pgvector/pgvector.

[8] Amazon Web Services, “AWS Documentation: AWS Lambda, Amazon API Gateway, Amazon S3, Amazon SQS, Amazon RDS, Amazon SES, Amazon ECR, Amazon CloudWatch and IAM.” [Online]. Available: https://docs.aws.amazon.com/.

[9] React, “React Documentation,” React. [Online]. Available: https://react.dev/

[10] Docker, “Docker Documentation.” [Online]. Available: https://docs.docker.com/.

[11] HashiCorp, “Terraform Documentation.” [Online]. Available: https://developer.hashicorp.com/terraform/docs.

[12] GitHub, “GitHub Actions Documentation.” [Online]. Available: https://docs.github.com/actions.

[13] Hugging Face , “Sentence-Transformers: all MiniLM-L6-v2.” [Online]. Available : https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2